



PLATFORM WALKTHROUGH

How this outreach + CRM platform works, and what you need to set up

Handover guide for the team

May 2026

Contents

01 What this is

02 Tech stack

03 The CRM pipeline

04 Outreach automation

05 The AI inbox

06 Cal.com to CRM sync

07 Webhooks and integrations

08 What you need to configure

09 Deploying on Railway

10 First-run checklist

01

WHAT THIS IS

The short version

This is a self-hosted sales platform. It runs cold-email campaigns, watches the replies, classifies them with AI, and pushes the results into a CRM pipeline. When someone books a call through Cal.com, that booking automatically advances the deal to the next stage. The whole thing lives behind a single admin login, deploys to Railway as two services (web + worker), and uses Postgres for everything including the background job queue.

You don't run separate tools for outreach, inbox, CRM, and booking. They are one system, sharing one database, with one source of truth per contact.

What you get out of the box

Outreach automation

Three-step email sequences, per-sender accounts, AI-personalised bodies, business-hours scheduling, deliverability throttling.

AI inbox

Inbound replies are scored for sentiment and intent. The AI drafts a suggested reply in your voice. You hit send or edit.

CRM pipeline

Contacts, deals, and stages (Lead, Contacted, Meeting Booked, Proposal, Won, Lost). Status hierarchy stops accidental downgrades.

Webhook plumbing

Cal.com bookings advance deals. Resend events update send/open/reply state. Inbound emails route to the right contact by Message-ID.

02

TECH STACK

What it's built on

You don't need to know all of this to run the platform, but it helps when something breaks.

Layer	Tool	What it does here
Runtime	Bun	Faster than Node for installs and scripts. The dev server and the worker both run on Bun.
Framework	Next.js 16 (App Router)	The admin UI and all API routes. React 19 on the front end.
Database	PostgreSQL	Single Postgres instance holds CRM data, outreach state, jobs queue, and auth sessions.
ORM	Drizzle	Typed SQL builder. Schema in <code>lib/db/schema.ts</code> , migrations in <code>lib/db/drizzle/</code> .
Job queue	pg-boss	Background jobs (sending emails, processing replies) live in a <code>pgboss</code> schema in the same Postgres.
Email send	Resend	Sends outbound mail, posts webhooks back when mail is delivered, opened, clicked, bounced, or replied to.
AI	Anthropic Claude	Classifies inbound replies (sentiment, intent) and drafts suggested responses.
Auth	NextAuth v5	Email + password with TOTP two-factor. Admin-only, the public site redirects to <code>/admin</code> .
Storage	S3-compatible	MinIO for local dev, Railway Bucket (Tigris) for production. Holds avatars and logos.

The two-service shape

One repo, two long-running processes on Railway:

website

The Next.js app. Serves the admin UI, the API routes, and the webhook endpoints. Built from `Dockerfile`.

worker

Bun process polling pg-boss. Sends scheduled outreach emails, processes inbound replies, runs cron jobs. Built from `Dockerfile.worker`.

Why two services: the web server can't run long jobs without blocking requests. The worker is the same code with a different start command. They share the database; the web side writes job rows, the worker reads and runs them.

03

THE CRM PIPELINE

Contacts and deals

Every person you talk to is a **contact**. Every active opportunity attached to a contact is a **deal**. One contact can have many deals over time (think "first project", "follow-up project"). At any moment, a contact has a **status** and each of their deals has a **stage**.

The pipeline stages

Stage	How a deal lands here
Lead	Contact form submitted, or contact imported into an outreach campaign.
Contacted	You sent them an email and they replied positively (AI flagged it).
Meeting Booked	They booked a call through Cal.com. Only the Cal.com webhook can advance to this stage , no AI shortcut.
Proposal Sent	Moved by you, manually, after the meeting.
Won	Closed deal. Manual.
Lost	Dead deal. Manual.

Status hierarchy (never downgrades)

A contact's status follows a strict ladder:

1	subscriber	Signed up for the newsletter, no sales contact yet.
2	lead	You started an outreach conversation, or they filled the contact form.
3	qualified	They booked a call through Cal.com.
4	customer	Closed deal.

Why this matters: if a customer replies to a newsletter, the system won't accidentally write them back to "subscriber". The protection lives in the database function `upsert_contact_with_hierarchy_protection` .

04

OUTREACH AUTOMATION

How a campaign is structured

A campaign is a bucket of contacts, a sender account, and three email steps. Each step has a delay (e.g. "send email 2 three business days after email 1"), a subject template, and a body template. The body template wraps the AI-personalised paragraph for that contact.

You write the template once at the campaign level. The contact-specific content (their first name, their company, the AI-generated opener) gets stitched in at send time.

Tokens you can use in templates

Token	Resolves to
<code>{{email_1_body}}</code>	The AI-personalised body paragraph for step 1, written per contact.
<code>{{email_2_body}}</code> / <code>{{email_3_body}}</code>	Same idea, for steps 2 and 3.
<code>{{signature}}</code>	The sender account's signature (HTML or plain text).
<code>{{unsubscribe_link}}</code>	System-rendered styled unsubscribe footer. Only appears if the token is present, so it's WYSIWYG.
<code>{{first_name}}</code>	Contact's first name.
<code>{{unsubscribe_url}}</code>	Raw URL form, for hand-rolled footers.

Sender accounts

Each campaign sends from a **sender account**. A sender account is a real mailbox (e.g. `jake@email.example.com`) registered with Resend, with its own signature, daily send cap, and warmup state. Campaigns can rotate across multiple senders to spread volume.

Reply-To rule (locked): outgoing outreach emails always set `Reply-To` to the sender's plain mailbox, never a UUID-suffixed address. Recipients see a real human address when they hit reply. Replies are still matched to the right contact via the `Message-ID` header in the inbound webhook.

Scheduling and business hours

The worker calculates each email's send time using:

- The step's delay in business days (configured per campaign).
- The campaign's business-hours window (e.g. 09:00 to 17:00 Australia/Sydney).
- The sender account's daily cap, so a single mailbox doesn't blast 500 in one hour.

If a send would land outside business hours, it gets pushed to the next open window. Jobs sit in pg-boss until their scheduled time, then the worker picks them up.

05

THE AI INBOX

What the AI does

Every inbound reply hits a Resend webhook, which lands the message in the database against the right contact. Before it shows up in the admin inbox, the AI analyser runs and tags it with four things:

Field	What it means
Sentiment	<code>positive</code> , <code>negative</code> , or <code>neutral</code> . The headline read on the reply.
Intent	One of: <code>schedule_call</code> , <code>wants_info</code> , <code>objection</code> , <code>future_followup</code> , <code>not_interested</code> , <code>unsubscribe</code> , <code>other</code> . More precise than sentiment.
Summary	One-line plain-English summary so you can scan the inbox without opening every thread.
Suggested reply	A drafted response in your voice. You can send as-is, edit, or ignore.

How accurate is "positive"?

The AI reads the whole thread, not just the latest reply. If the prior turns establish context ("we talked last month, send me the proposal"), it factors that in. The sentiment field is the trigger that pushes a CRM update: a positive reply moves a deal from **Lead** to **Contacted**.

Important guardrail: a positive reply never auto-promotes a deal past **Contacted**. The AI can flag intent `schedule_call` , but only the real Cal.com booking webhook can move a deal to **Meeting Booked**. This stops "yes I'd love to chat" from being misread as a booking.

The voice (locked)

The suggested-reply prompt is tuned to a specific brand voice: Australian English, plainspoken, no em-dashes, no buzzwords. Banned phrases include "AI automation", "AI-powered", "circle back", "touch base", "reach out", "leverage", "synergy". The signoff is fixed format with the sender's first name injected.

If you change the prompt in `lib/outreach/ai/reply-analyzer.ts` , run `bun scripts/eval-replies.ts` to sanity-check the output against eight test cases before shipping.

06

CAL.COM TO CRM SYNC

The booking flow

1. A contact replies positively to an outreach email. The AI flags it; the deal moves to **Contacted**; the suggested reply includes your Cal.com link.
2. You send the reply (or the AI's suggestion auto-fills it). The contact clicks the Cal.com link.
3. The contact books a slot. Cal.com fires a `BOOKING_CREATED` webhook to your platform.
4. The webhook handler verifies the HMAC signature, finds the contact by email, upgrades their status to `qualified`, and moves the deal to **Meeting Booked**.
5. The booking appears in the contact's timeline, with the start time and meeting link.

Triggers Cal.com should send

BOOKING_CREATED	Contact → qualified. Deal → Meeting Booked. Booking saved to timeline.
BOOKING_RESCHEDULED	Booking time updated on the existing timeline entry.
BOOKING_CANCELLED	Timeline marked cancelled. Deal stage does NOT auto-revert (manual call).

DNS gotcha

If you put your site behind Cloudflare, do not point the Cal.com webhook at your apex domain. Cloudflare's free-tier Bot Fight Mode returns HTTP 403 for Cal.com's POSTs before they reach your server. The fix is to set up a separate `hooks.yourdomain.com` subdomain as DNS-only (grey cloud) pointing at your Railway service, and use that as the Cal.com subscriber URL. The existing setup follows this pattern; don't "clean it up" by collapsing back to one domain.

07

WEBHOOKS AND INTEGRATIONS

Every webhook the platform owns

Endpoint	Source	Purpose
/api/webhooks/cal	Cal.com	Booking lifecycle, drives CRM stage changes.
/api/outreach/webhooks/resend	Resend	Send status, opens, clicks, bounces, inbound replies. Drives campaign analytics and the AI inbox.

How inbound replies match the right contact

When Resend posts an inbound reply, the webhook handler:

1. Pulls the `In-Reply-To` and `References` headers from the email.
2. Looks for those Message-IDs in `outreach_contacts.email_{1,2,3}_resend_id`. (Resend uses the original send ID as the Message-ID.)
3. If headers are missing (some mail clients strip them), falls back to matching the inbound From address.
4. Files the reply against that contact and triggers the AI analyser.

This is why you should never reintroduce UUID-suffixed Reply-To addresses, the contact-matching path is what makes the brand-friendly Reply-To safe.

Security on every webhook

Both endpoints verify an HMAC-SHA256 signature against a shared secret. If the signature is wrong the request is rejected with 401. Both endpoints also de-duplicate by event/message ID, so a retry from Cal.com or Resend can't double-process the same event.

08

WHAT YOU NEED TO CONFIGURE

The minimum env var set

Copy `.env.example` to `.env.local` for local dev, and set these in Railway → Variables for production. Every one of these must be filled or the app will fail to boot.

Variable	What it's for
<code>DATABASE_URL</code>	Postgres connection string. Use Railway's public URL for local dev.
<code>AUTH_SECRET</code>	NextAuth session signing key. Generate with <code>openssl rand -base64 32</code> .
<code>NEXTAUTH_URL</code>	Public URL of the admin (e.g. <code>https://admin.yourdomain.com</code>).
<code>INTEGRATION_ENCRYPTION_KEY</code>	Encrypts API keys stored in the DB. <code>openssl rand -hex 32</code> . Do not change this once data exists.
<code>RESEND_API_KEY</code>	From your Resend dashboard. Outbound + inbound email.
<code>ANTHROPIC_API_KEY</code>	From console.anthropic.com. Powers the AI reply analyser.
<code>DEFAULT_FROM_EMAIL</code>	The verified sending address (e.g. <code>hello@email.yourdomain.com</code>).
<code>NEWSLETTER_FROM_EMAIL</code>	Separate from-address for newsletter sends if you want one.
<code>CAL_WEBHOOK_SECRET</code>	Generate, then paste into Cal.com's webhook settings as the shared HMAC secret.
<code>UNSUBSCRIBE_SECRET</code>	Signs unsubscribe links so they can't be forged.
<code>BUCKET_*</code>	Six vars for S3-compatible storage (endpoint, region, name, force-path-style, access key, secret key).

One-off identity setup

Even after env vars are set, you need to do this per-tenant:

1. **Verify your sending domain in Resend** (SPF, DKIM, DMARC). Without this, mail will go to spam or be rejected outright.
2. **Create at least one sender account** in the admin (Outreach → Sender Accounts). Give it a real mailbox, a display name, and a signature.
3. **Set up a Cal.com event type** for your discovery / consultation call. Copy its booking URL into the AI reply analyser's system prompt so suggested replies link to the right URL.
4. **Register the Cal.com webhook** in Cal.com's settings → webhooks. URL is `https://hooks.yourdomain.com/api/webhooks/cal`; secret matches `CAL_WEBHOOK_SECRET`; triggers include `BOOKING_CREATED` at minimum.
5. **Register the Resend webhook** in Resend's dashboard. URL is `https://yourdomain.com/api/outreach/webhooks/resend`; subscribe to send, delivered, opened, clicked, bounced, complained, and email.received.
6. **Create your admin user** via the invitation flow (Settings → Users) and set up TOTP two-factor on first login.

09

DEPLOYING ON RAILWAY

The three services

One Railway project, three services, all from this same repo:

Service	Builder	Start command
website	Dockerfile	<code>node .next/standalone/server.js</code>
worker	Dockerfile.worker	<code>bun scripts/worker.ts</code>
Postgres	Railway managed	(not your problem)

Both services deploy from the same **main** branch. The build configs in `railway.json` and `railway.worker.json` tell Railway which Dockerfile each service should use. Push to main, both services rebuild.

Add an object-storage bucket

Avatars and uploaded logos go to S3-compatible storage. In Railway, add a Bucket service to the project (it's a one-click resource). Then copy its credentials into the website service's `BUCKET_*` env vars. For local development, run `docker compose up -d minio`, the bucket is created automatically and the `BUCKET_*` defaults in `.env.example` already point at it.

Database migrations

Schema migrations live in `lib/db/drizzle/*.sql`. They're plain SQL files, run in order. To apply them against a database:

```
bunx drizzle-kit migrate
```

When you change `lib/db/schema.ts`, generate a new migration file with:

```
bunx drizzle-kit generate
```

Never run `drizzle-kit push` against production. Push bypasses migration files and can corrupt the live DB. Push is for local-dev iteration only; migrate is for everything else.

10

FIRST-RUN CHECKLIST

From zero to sending

1. Clone the repo, `bun install`.
2. Copy `.env.example` to `.env.local`, fill in `DATABASE_URL` + `AUTH_SECRET` + the keys from section 8.
3. `docker compose up -d minio` to start local object storage.
4. `bunx drizzle-kit migrate` to create the schema.
5. `bun run dev`. The admin boots on `http://localhost:3500`.
6. Hit the login page, create your admin via the invitation flow, set up TOTP.
7. In Settings → Integrations, paste in your Resend, Anthropic, and Cal.com keys.
8. In Outreach → Sender Accounts, create your first sender mailbox with a signature.
9. In Outreach → Campaigns, create a campaign, write the three step templates, import contacts (CSV).
10. Start the campaign. The worker picks up the first batch of sends inside the next business-hours window.

When something breaks

Emails not sending

Check the worker logs in Railway. The web service can enqueue jobs but only the worker sends them. If the worker is down, the queue grows.

Replies not appearing

Check the Resend webhook is configured and that `email.received` is enabled. Verify the HMAC secret matches `RESEND_WEBHOOK_SECRET` in `env`.

Bookings not updating the CRM

Cal.com webhook is the only path. Check the `hooks.` subdomain is grey-cloud in Cloudflare, not orange-cloud. Verify `CAL_WEBHOOK_SECRET` matches.

AI suggesting weird replies

Run `bun scripts/eval-replies.ts` to see what the prompt produces against eight known inputs. If a phrase keeps creeping in, add it to the banned list in `reply-analyzer.ts`.

One sentence to remember

The website enqueues, the worker sends, the AI reads, the webhooks decide. Each piece does one thing; they meet in Postgres.